

System Requirements Analysis and Design Documentation

Price Comparison Application

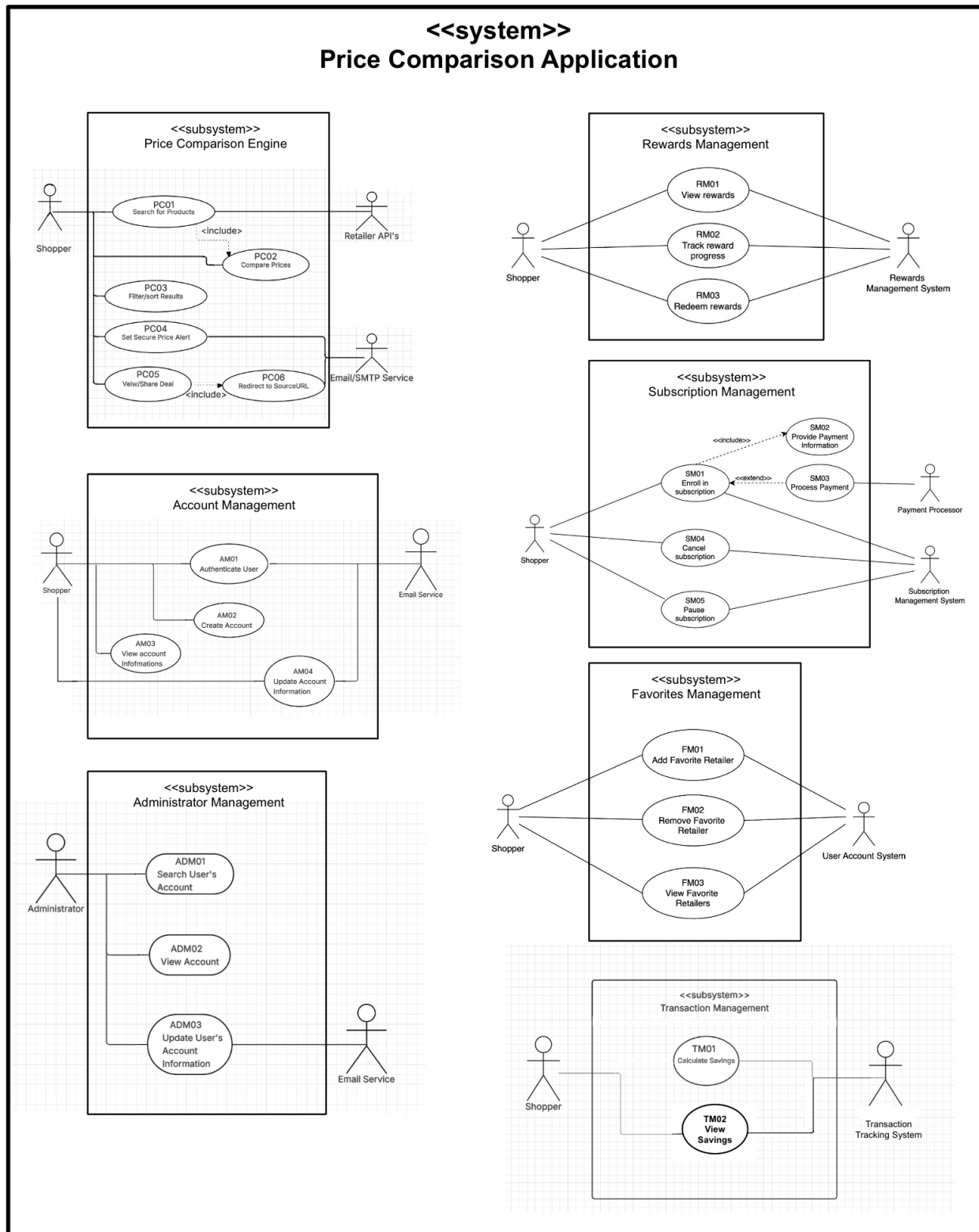
Group 3:

Rose Garcia, Chase Okoorian, Elizabeth Philip, and Xavier Le

Table of Contents

Use Case Diagram.....	3
Use Case Descriptions.....	4-15
Class Diagram.....	16
Activity Diagrams.....	17-18
Sequence Diagrams.....	19-21
Appendix.....	22-28

Use Case Diagram



Use Case Descriptions

Use Case Name: View Rewards
Rationale: It would be beneficial to further describe the system-user interactions for this use case to ensure all requirements are uncovered so app users don't encounter any issues viewing their rewards.
Reason: I selected this use case because earning and then viewing rewards is a major function of the app. The value offered is an app that lets users compare prices across retailers in one centralized location and earn rewards. Users need to be able to see which rewards they have earned to continue providing this value.
Primary Actor(s): Shopper
Secondary Actor(s): Rewards Management System
Pre-conditions: 1. The shopper is logged into the application. 2. Affiliate transactions have been verified. 3. Points balance has been updated and saved. 4. Rewards data is available.
Post-conditions: 1. The shopper can view available rewards. 2. The shopper can view locked rewards. 3. The shopper can see reward details and current status.
Main Flow: 1. The shopper navigates to the rewards page. 2. The app requests the shopper's rewards data. 3. The Rewards Management System provides rewards information. 4. The app displays all available and locked rewards, and points required for each. 5. If the shopper selects a reward, the application displays its details, including reward status, type, and redemption details. 6. Use case ends.
Alternate Flow: 1. At step 6, the shopper has no available rewards. 2. The app displays only locked rewards or a message indicating that no rewards are currently available. 3. The use case ends.
Exception Flow: 1. At step 3, the app cannot retrieve reward data. 2. The app displays an error message indicating that rewards cannot be loaded at this time. 3. The shopper cannot view rewards. 4. The use case ends.

Use Case Name: Enroll in Subscription
Rationale: The app is subscription-based. We must ensure that users can successfully enroll in a subscription to use the app.
Reason: This use case was selected because subscription enrollment is a core feature of the application. If enrollment fails, users cannot gain full access to the app.
Primary Actor(s): Shopper
Secondary Actor(s): Payment Processor, Subscription Management System
Pre-conditions: 1. The shopper has created an account. 2. The shopper is logged into the app. 3. The shopper does not already have an active subscription.
Post-conditions: 1. The subscription record is saved. 2. The payment record is saved. 3. The shopper is granted full access to the application.
Main Flow: 1. The shopper selects to enroll in a subscription to access the app. 2. The app displays subscription plans and enrollment details. 3. The shopper selects a subscription plan. 4. The app requests payment information. 5. The shopper enters payment information. 6. The application sends the payment information to the Payment Processor. 7. The Payment Processor processes and approves the payment. 8. The subscription management system creates and stores the new subscription. 9. The app displays enrollment confirmation to the shopper. 10. The app grants full access. 11. Use case ends.
Alternate Flow: 1. At step 3, the shopper selects a free trial plan. 2. The app requests payment information for future billing after the trial ends. 3. The shopper enters payment information. 4. The Subscription Management System creates and stores the new subscription with a free trial status. 5. The use case resumes at step 9.
Exception Flow: 1. At step 8, the Subscription Management System fails to save the subscription record. 2. The shopper tries to enter the app again later. 3. The application displays an error message indicating that the enrollment was not successful. 4. Full access to the application is not granted. 5. The use case ends.

Use Case Name: Update Account Information
Rationale: The app is subscription-based, billed monthly. We need to ensure that all users can update their personal information, such as billing and payment details, to maintain uninterrupted access to the app.
Reason: This use case was selected because maintaining up-to-date personal information is necessary in avoiding paused or cancelled subscriptions due to billing errors.
Primary Actor(s): Shopper
Secondary Actor(s): Database System
Pre-conditions: 1. The shopper has created an account. 2. The shopper is logged in to the app. 3. Personal information, such as name, email address, billing/mailling addresses, and payment information, is saved.
Post-conditions: 1. The updated personal information has been saved in the Database System. 2. The app displays updated information.
Main Flow: 1. The shopper navigates to their account details. 2. The app displays shopper account details, such as name, email address, billing address, and payment information. 3. The shopper edits the desired field with new information. 4. The app sends the information to the Database System. 5. The Database System verifies and saves the updated shopper record. 6. The app displays a message indicating the information has been successfully saved. 7. The app displays the shopper's updated information. 8. The use case ends.
Alternate Flow: 1. At step 3, the shopper enters invalid information. 2. The app displays an error message indicating that the information is invalid. 3. The shopper re-enters the information. 4. The use case resumes at step 4.
Exception Flow: 1. At step 5, the Database System fails to save the updated information. 2. The shopper tries to save their information again. 3. The app displays an error message indicating that the information was not saved. 4. The shopper's personal information was not saved. 5. The use case ends.

Use Case Name: View Savings History
Rationale: This use case is a feature of the application that helps users save money by comparing prices across different retailers.
Reason: It demonstrates the system's value and involves interaction with multiple components, making it important for system functionality.
Primary Actor(s): Shopper, Administrator
Secondary Actor(s): Retailer API, Transaction Tracking System, Social Media Integration
Pre-conditions: 1. User is logged into the system 2. Retailers already exist in the system
Post-conditions: 1. Retailer is added or removed from favorites 2. Favorites list is updated
Main Flow: 1. User navigates to a retailer page 2. User selects "Add to Favorites" 3. System stores retailer in the favorites list 4. User accesses "Favorites" section 5. System displays saved retailers 6. User may remove a retailer 7. System updates the list
Alternate Flow: 1. Alternate flow: If retailer is already in favorites, then system outputs message, "Retailer already saved!"
Exception Flow: 1. Exception flow: If there is a system error, then it displays the specific error message, and the action fails

Use Case Name: View Transaction Logs
Rationale: This use case allows users and administrators to review historical transactions. It helps track spending, monitor savings, and resolve disputes.
Reason: It demonstrates the system's value by providing transparency and accountability, and it involves interaction with multiple components such as transaction storage and reporting systems.
Primary Actor(s): Shopper, Administrator
Secondary Actor(s): Transaction Tracking System, Retailer API, Notification System
Pre-conditions: 1. User is logged into the system 2. Transactions have been recorded in the system
Post-conditions: 1. Transaction history is displayed to the user 2. Transactions can be filtered, sorted, or exported as needed
Main Flow: 1. User navigates to the "Transaction Logs" section. 2. System retrieves the user's transaction history from the Transaction Tracking System. 3. System displays a list of transactions with details (date, retailer, amount, savings, etc.). 4. User may filter transactions by date, retailer, or type. 5. User may sort transactions by amount, date, or retailer.
Alternate Flow: 1. If no transactions exist, the system displays a message: "No transactions found." 2. If the user applies filters that yield no results, the system displays: "No transactions match your criteria."
Exception Flow: 1. If the Transaction Tracking System is unavailable, the system displays an error message: "Unable to retrieve transaction history. Please try again later." 2. If data retrieval fails for a specific transaction, the system displays a warning for that transaction while showing others.

Use Case Name: Share Deals
Rationale: This use case allows users to share deals or discounts they find with friends or on social media, enhancing engagement and promoting the system's value.
Reason: Sharing deals increases user interaction, spreads awareness of discounts, and may drive more traffic to retailers, involving multiple system components such as social media integration and notification systems.
Primary Actor(s): Shopper
Secondary Actor(s): Social Media Integration, Notification System, Retailer API
Pre-conditions: 1. User is logged into the system 2. Deals exist in the system that can be shared
Post-conditions: 1. Selected deal is shared with the chosen recipients or platforms 2. System records the share activity for analytics and tracking
Main Flow: 1. User navigates to a deal or discount in the system. 2. User selects the "Share" option. 3. System displays sharing options (social media, email, messaging). 4. User selects the desired sharing method and optional message. 5. System sends the deal to the selected recipients or platform. 6. System confirms successful sharing to the user. 7. System logs the share activity for tracking and analytics.
Alternate Flow: 1. If the user cancels before sharing, the system returns to the deal page without logging the share. 2. If the deal has expired, the system displays: "This deal is no longer valid and cannot be shared."
Exception Flow: 1. If the social media platform or messaging service is unavailable, the system displays an error: "Unable to share the deal at this time. Please try again later." 2. If the user's network connection fails, the system displays: "Network error. Deal sharing failed."

Use Case Name: View User Account
Rationale: Allow Administrators to view the details of the shopper's account
Reason: Users might have issues regarding their accounts – whether they forgot their passwords and have no other methods to regain them, or if their accounts get hacked – users can request administrators to chime in and access their account directly, and potentially make any changes the user wants
Primary Actor(s): Administrator
Secondary Actor(s): Database System
Pre-conditions: 1. Shopper is unable to log in to their account or 2. Shopper account is compromised 3. User has given Admin their credential ie. username
Post-conditions: 1. Shopper account is safe 2. Shopper has access to their account again
Main Flow: 1. Admin access users' accounts database 2. Admin inputs a specific username/email to filter/search 3. System checks for the user's credentials 4. Admin navigates through the user's account page 5. Admin checks and relays the credentials
Alternate Flow: 1. Should the account be compromised, Admin, with permission and instruction from the user, may change the credentials
Exception Flow: 1. Should the server go down or be in maintenance, there will be no access to the account at the moment

Use Case Name: Authenticate User
Rationale: A shopper can have access to an account in a secure manner
Reason: Linking an email to an account allows shoppers to have other ways to reach their account should problems arise
Primary Actor(s): Shopper
Secondary Actor(s): Email service
Pre-conditions: 1. Shopper created or is creating an account
Post-conditions: 1. Shopper will have access to an authenticator for their account
Main Flow: 1. User enters credentials 2. System send a confirmation link to the user's email for confirmation 3. User checks email for the confirmation link 4. User clicks the link and confirms their email 5. Link sends user to the home page
Alternate Flow: 1. Should User login, they may have the system send codes to their email to input for security 2. Should the user forget their password, they must enter their email, and the system will send them a password reset link
Exception Flow: 1. Network error: "Network Error. Unable to send email."

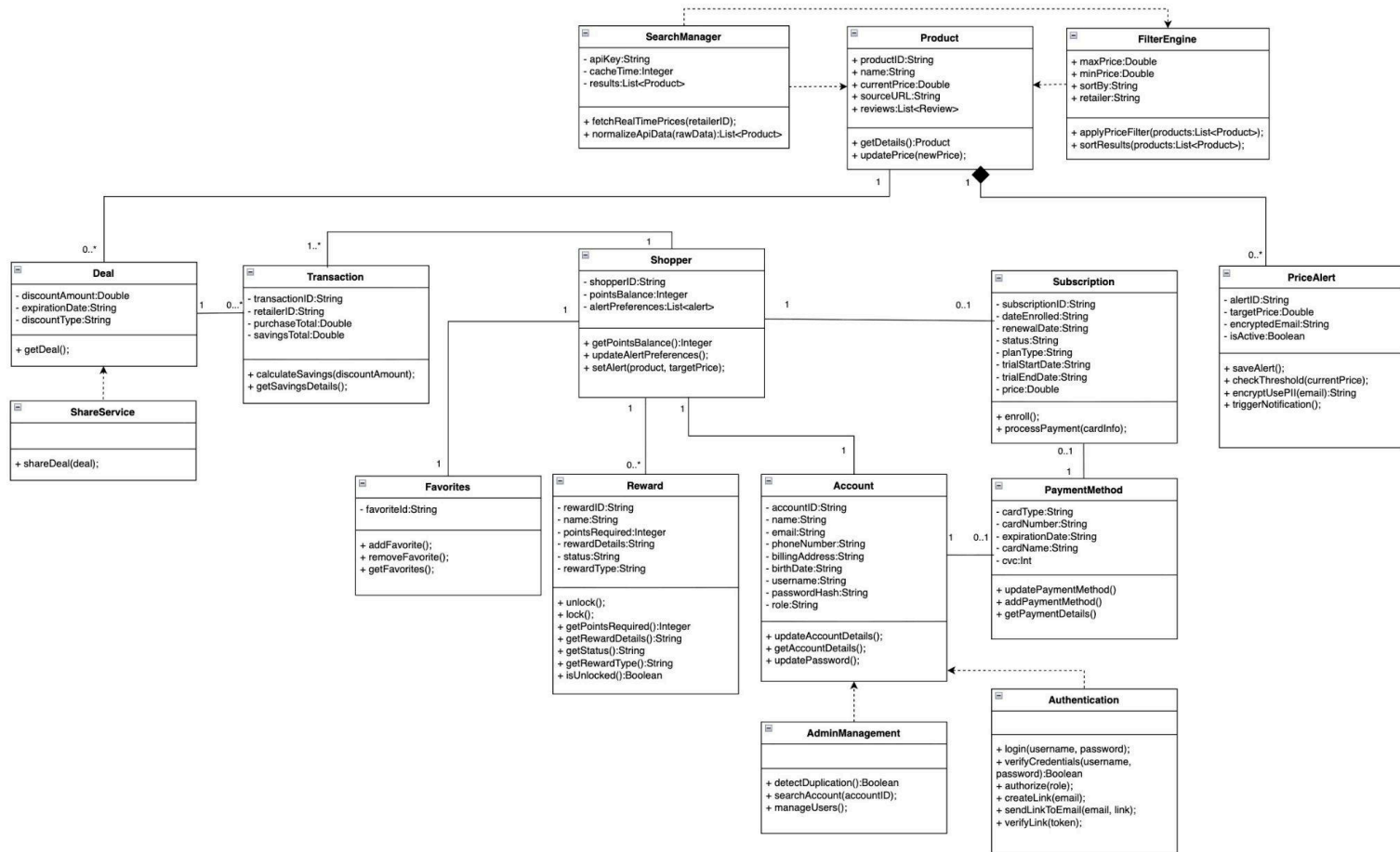
Use Case Name: Redirect to Source URL
Rationale: A shopper would want to access the direct retailer web to buy an item
Reason: Grant access to the original retailer webpage should a shopper want access to it; be it they have an account there already or out of preference
Primary Actor(s): Shopper
Secondary Actor(s): Retailer API
Pre-conditions: 1. Shopper has searched for an item
Post-conditions: 2. Shopper has access to the webpage of the retail item
Main Flow: 1. User compares the prices of different retailers 2. User find prices that fit their budget 3. User check for webpage of the retailer 4. User clicks the weblink 5. System transfer user over to retailer's website via the Retailer API
Alternate Flow: 1. Suppose instead of clicking the link, the User navigates back instead
Exception Flow: 1. Website link is down; "Website unavailable."

Use Case Name: Search for Product
Rationale: It represents the core engine of the application. It demonstrates How the system interacts with multiple APIs
Reason: It provides the core value of the system.
Primary Actor(s): Shopper, Administrator
Secondary Actor(s): Retailer APIs, Email/SMTP Service
Pre-conditions: 1. System has access to a stable internet connection. 2. Valid API keys are configured, not hard-coded.
Post-conditions: 1. A normalized list of products from various sources is displayed to the user. 2. The list is cached for 30 minutes
Main Flow: 1. Shopper enters the product name or model number in the search bar. 2. System initiates requests to integrated Retailer APIs (Amazon, Walmart, Etc.) 3. System receives raw data and maps it into a unified comparison table format. 4. System displays the results, including name, price, and source.
Alternate Flow: 1. System checks local data and finds identical search results timestamped within the last 30minutes. 2. System retrieves the stored data from cache instead of querying external APIs.
Exception Flow: 1. If the result set exceeds 100 items, the system sends a new request to the server to handle the filtering and sorting (per SR-002)

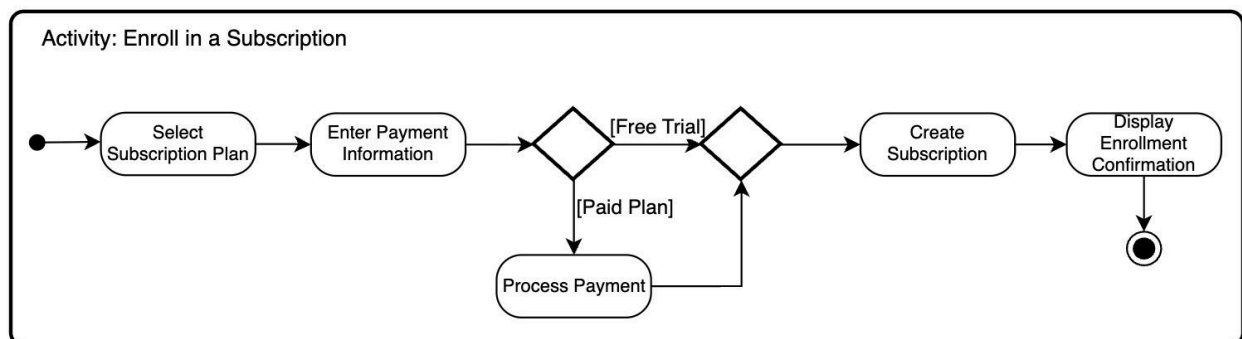
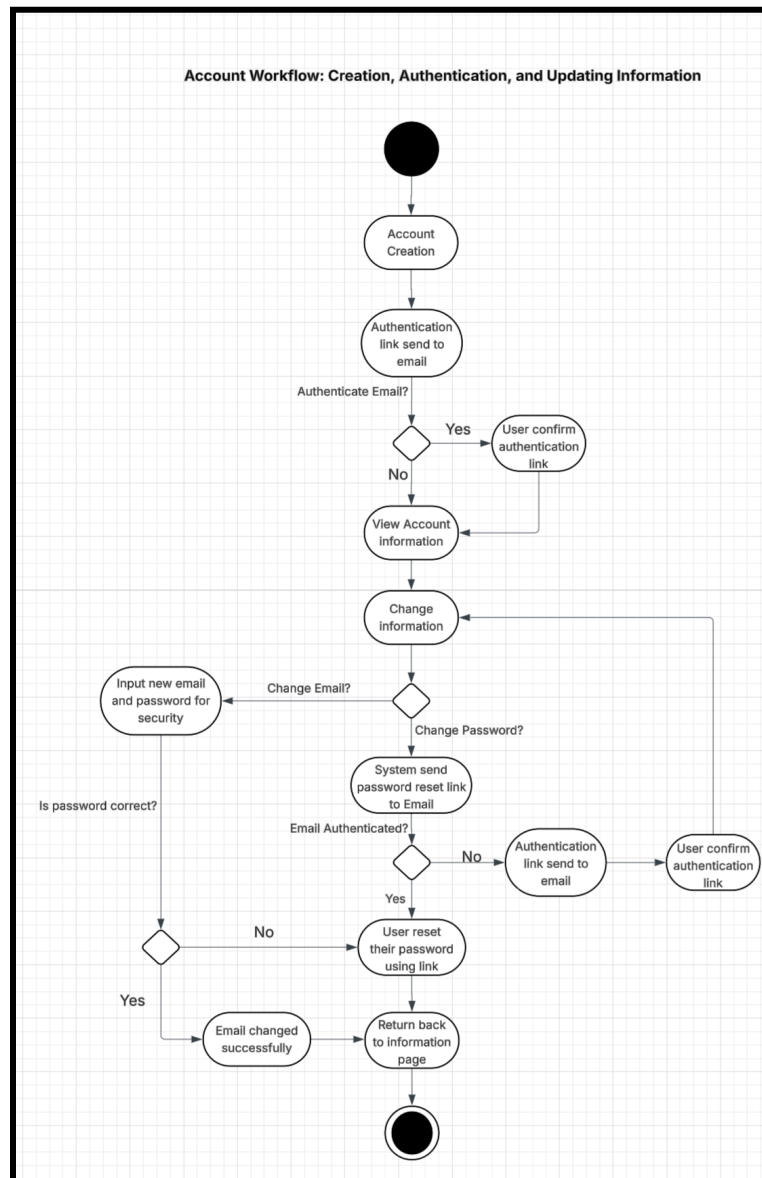
Use Case Name: Set Secure Price Alert
Rationale: It integrates our high-priority security requirement, encrypting User PII at rest using AES-256 standards.
Reason: it provides value to users by allowing them to be alerted to price changes on particular items of interest.
Primary Actor(s): Shopper
Secondary Actor(s): Email/SMTP Service
Pre-conditions: 1. Shopper is logged into a verified account. 2. Shopper has selected a specific product
Post-conditions: 1. A target price alert is saved. 2. User contact info is encrypted in the database.
Main Flow: 1. Shopper clicks “Set Alert” and enters a “Target Price”. 2. System validates the price output. 3. System calls the encryption module to secure the user’s email address. 4. System saves the alert and sends a confirmation to the Shopper.
Alternate Flow: 1. System compares the entered “Target Price” against the currentPrice attribute of the Product object 2. System identifies that the current market price is already equal to or lower than the requested target price. 3. System bypasses the encryption and database steps to prevent redundant data processing. 4. System displays a notification to the shopper, “The current price meets your target! Would you like to buy now?” and provides a direct link to sourceURL.
Exception Flow: 1. If the Shopper enters an invalid or non-numeric price, the system displays an “invalid input” error and clears the field.

Use Case Name: Filter/Sort Results
Rationale: Critical for how/who the handles system data and how the system represents that data.
Reason: Handles representing valuable information and the processing constraint between the client side and server-side based on the result size.
Primary Actor(s): Shopper
Secondary Actor(s): Retailer APIs
Pre-conditions: 1. A list of search results is currently displayed on the UI.
Post-conditions: 1. The displayed list is filtered by price range/brand on the UI. 2. Meets user preference/condition (ex. High to Low)
Main Flow: 1. Shopper adjusts the price slider to set a maximum price. 2. System checks the result set size; if under 100 items, it filters locally to improve responsiveness 3. Shoper selects the “Low to High” sorting option. 4. System reorders the product list starting with the lowest price.
Alternate Flow: 1. System refreshes the display table and resets sorting preferences.
Exception Flow: 1. If the result set exceeds 100 items, the system sends a new request to the server to handle the filtering and sorting(per SR-002)

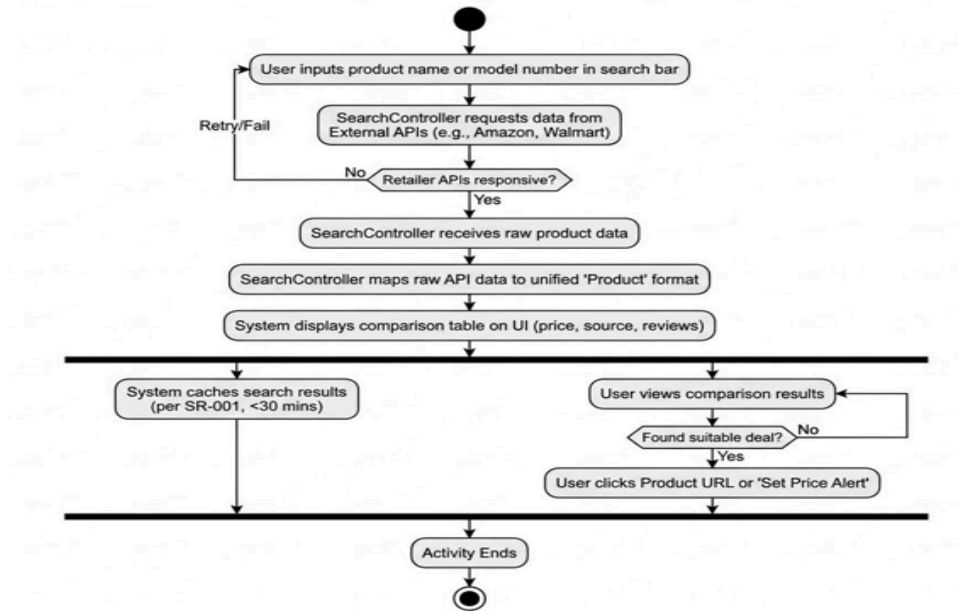
Class Diagram



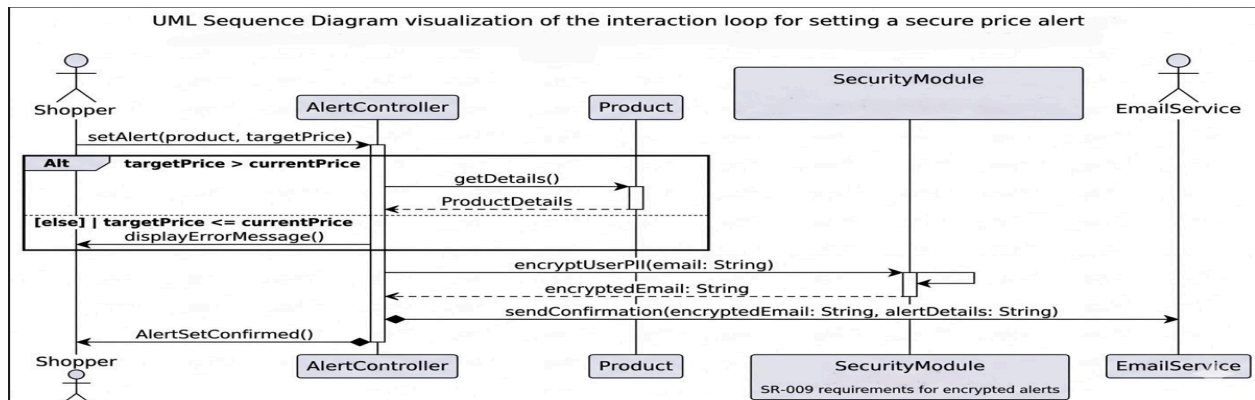
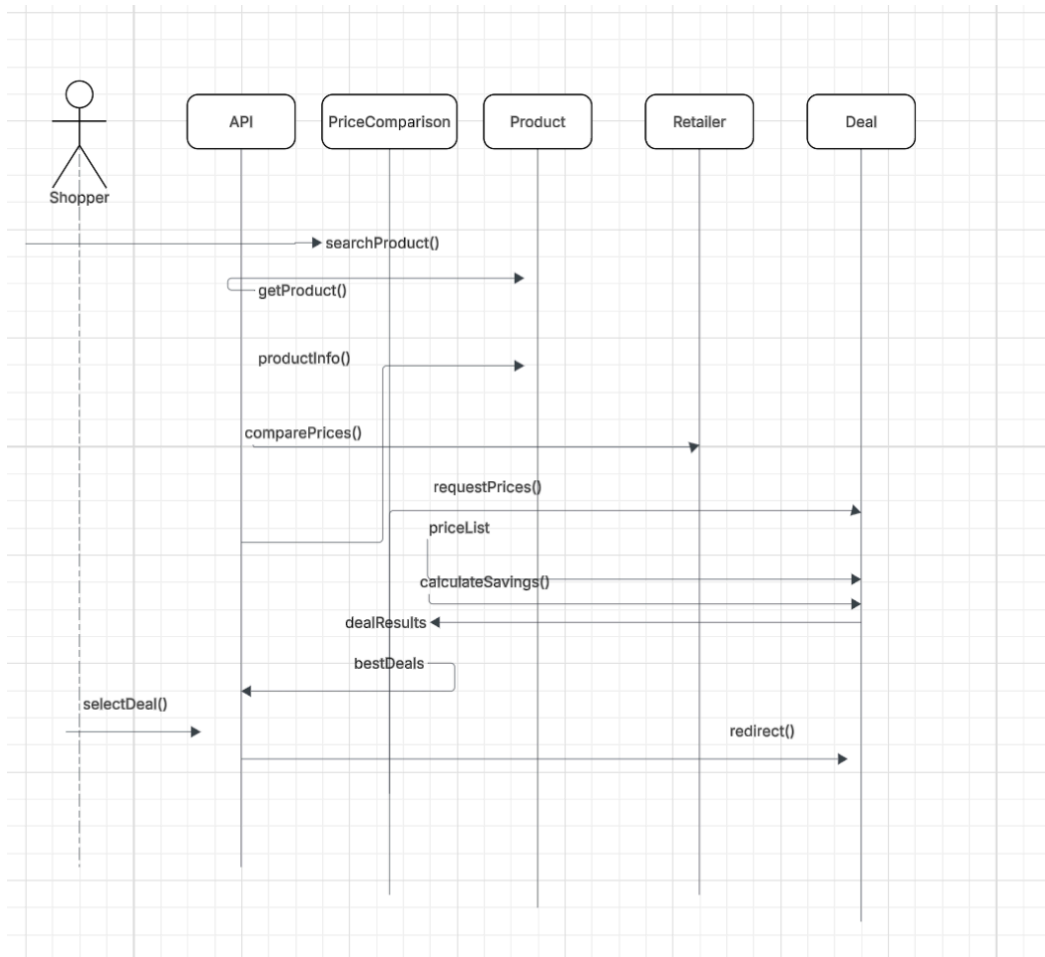
Activity Diagrams



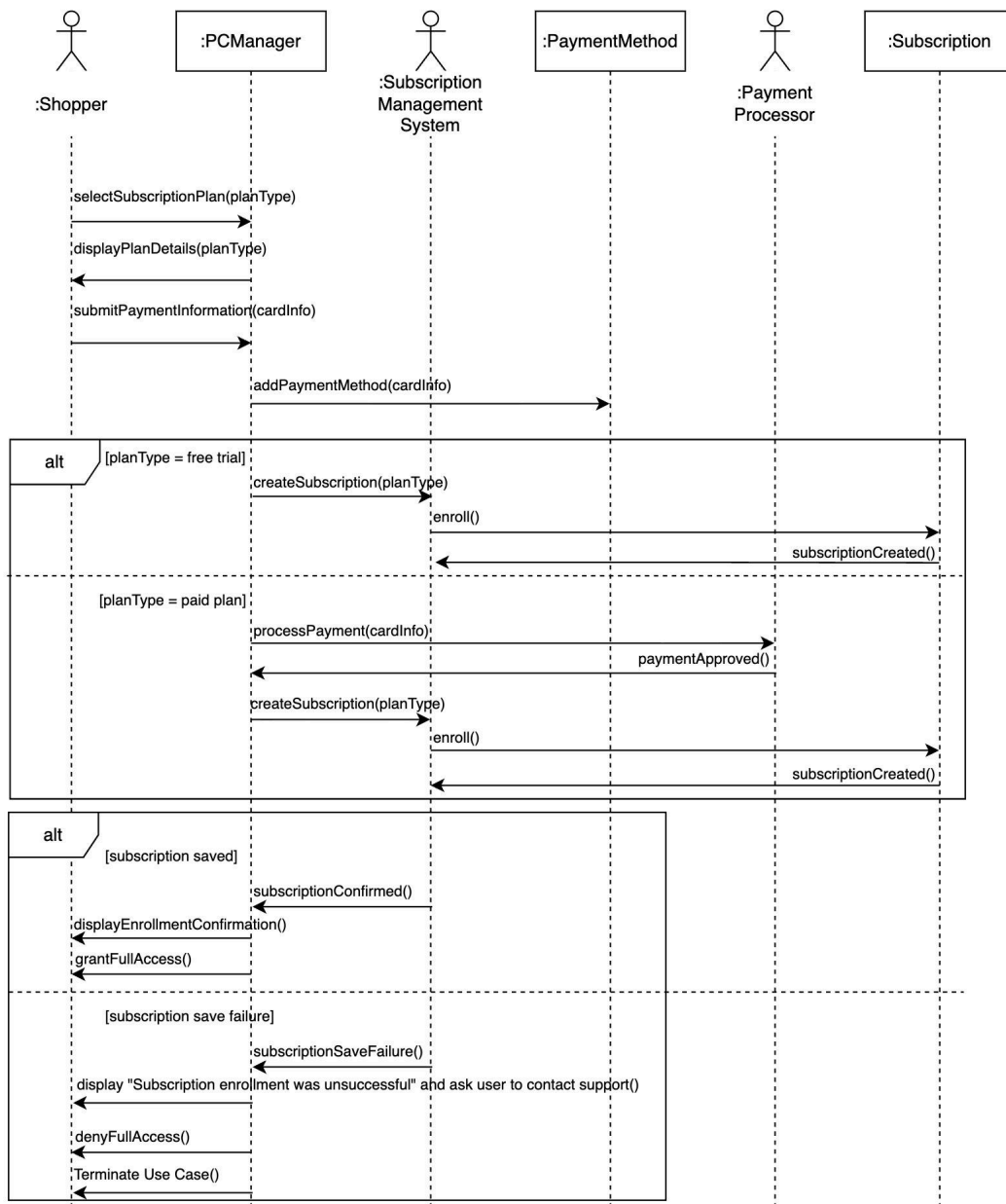
Search & Compare Products Workflow (UR-001)

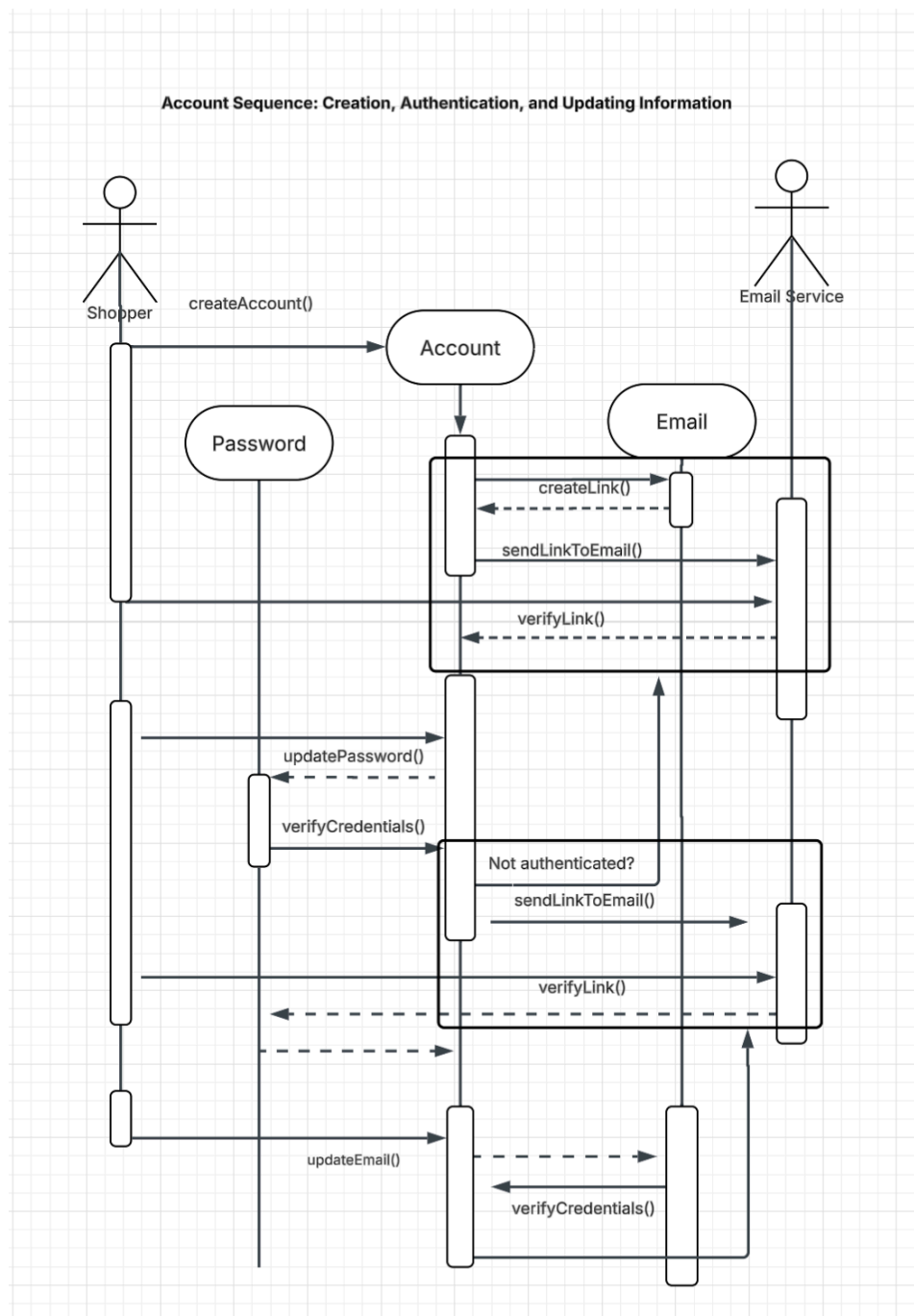


Sequence Diagrams



Sequence Diagram: Enroll in a Subscription



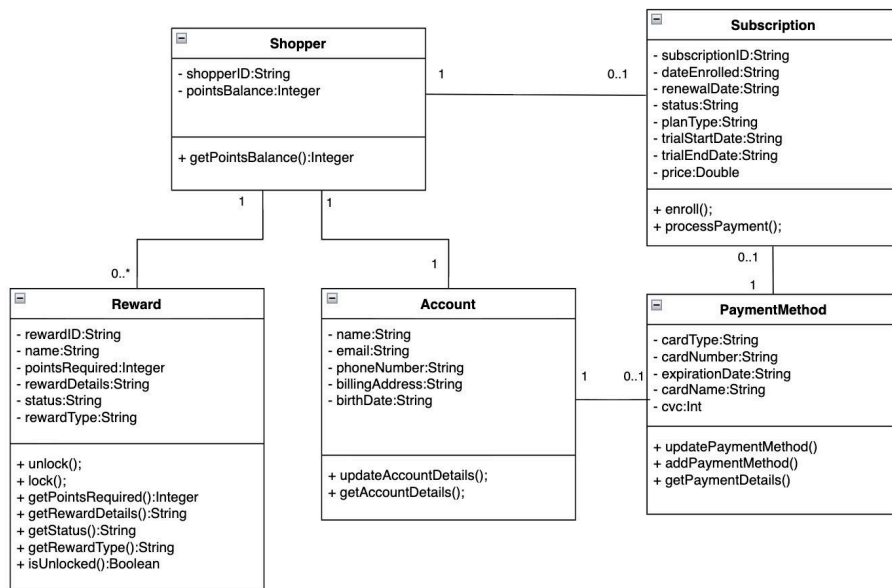


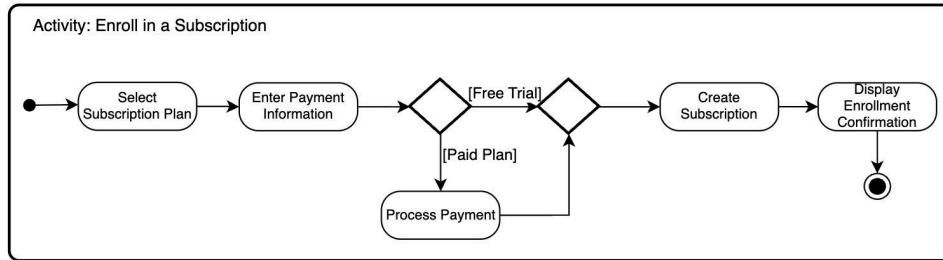
Appendix

Group Member Contributions

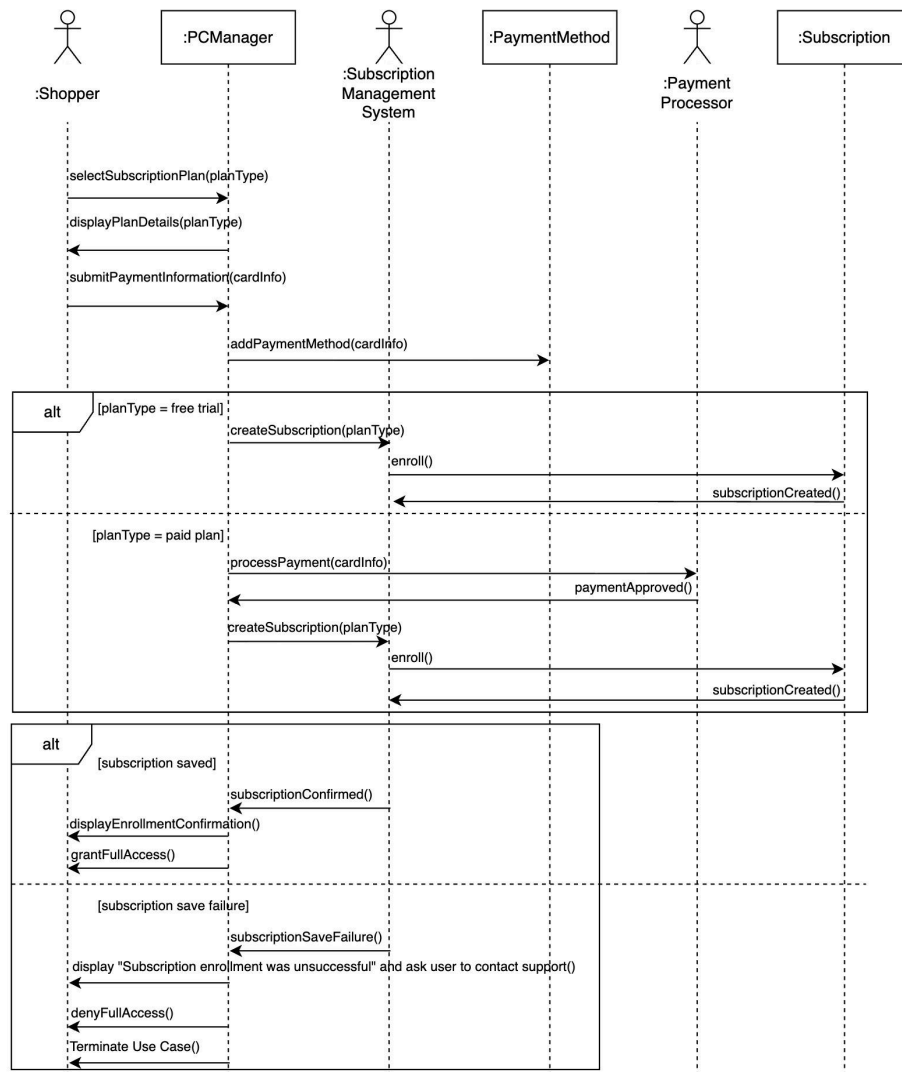
Rose Garcia:

- Diagrams:





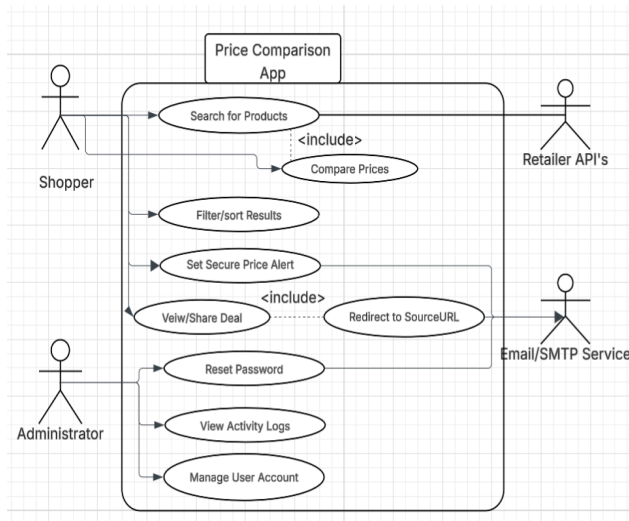
Sequence Diagram: Enroll in a Subscription



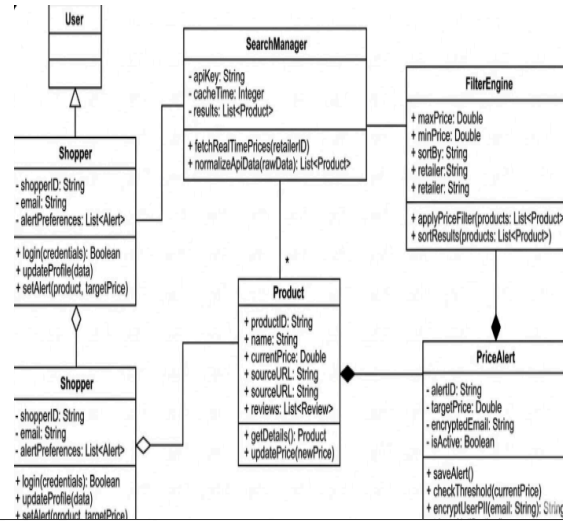
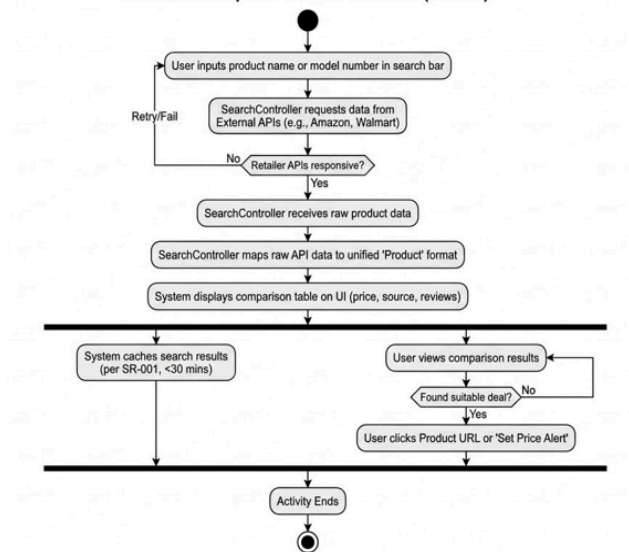
- **Use Case Descriptions:**
 - View Rewards
 - Enroll in Subscription
 - Update Personal Information

Chase Okoorian:

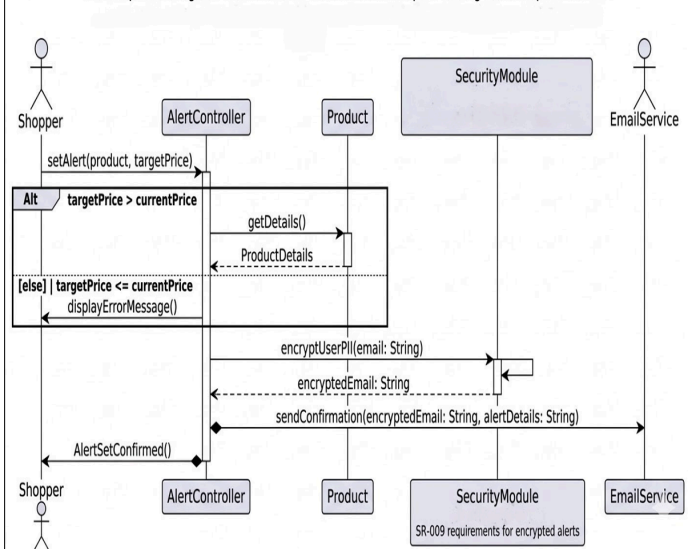
Diagrams:



Search & Compare Products Workflow (UR-001)



UML Sequence Diagram visualization of the interaction loop for setting a secure price alert

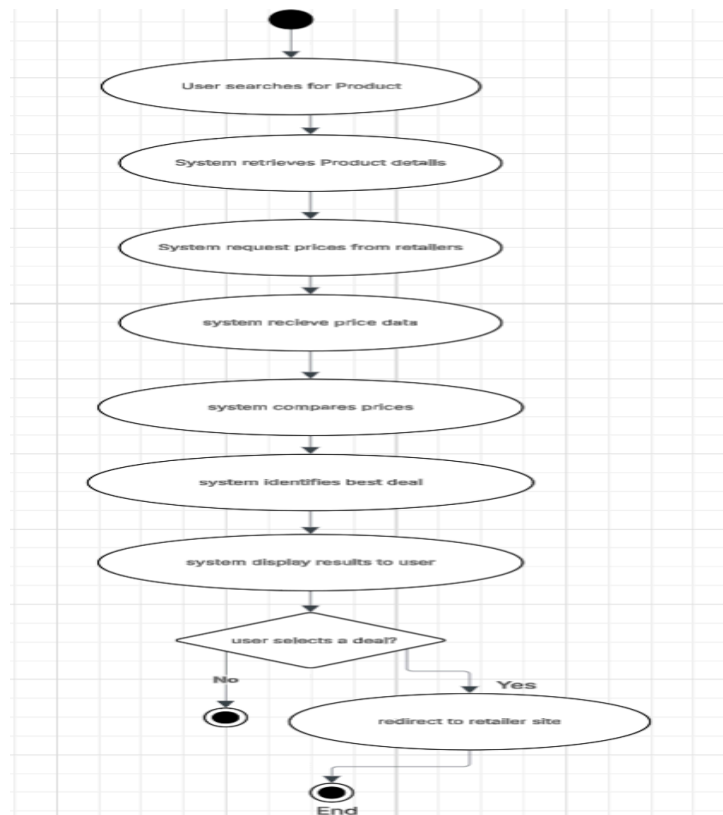
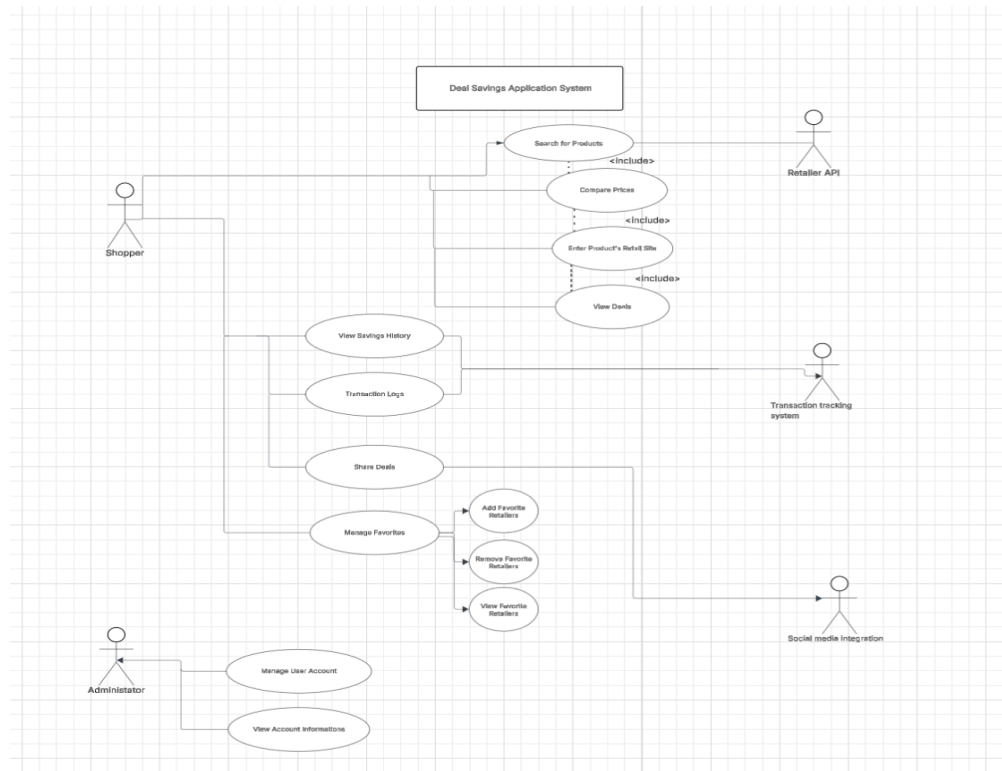


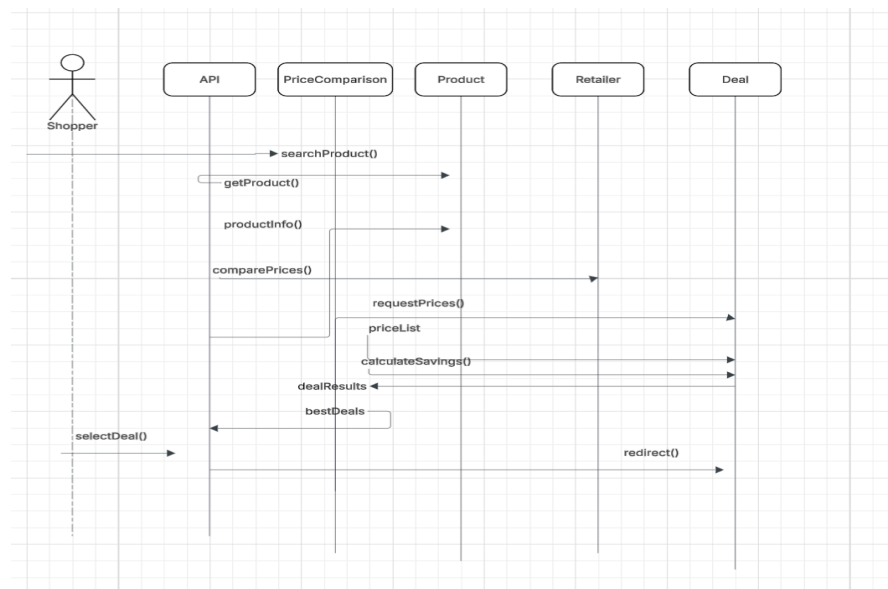
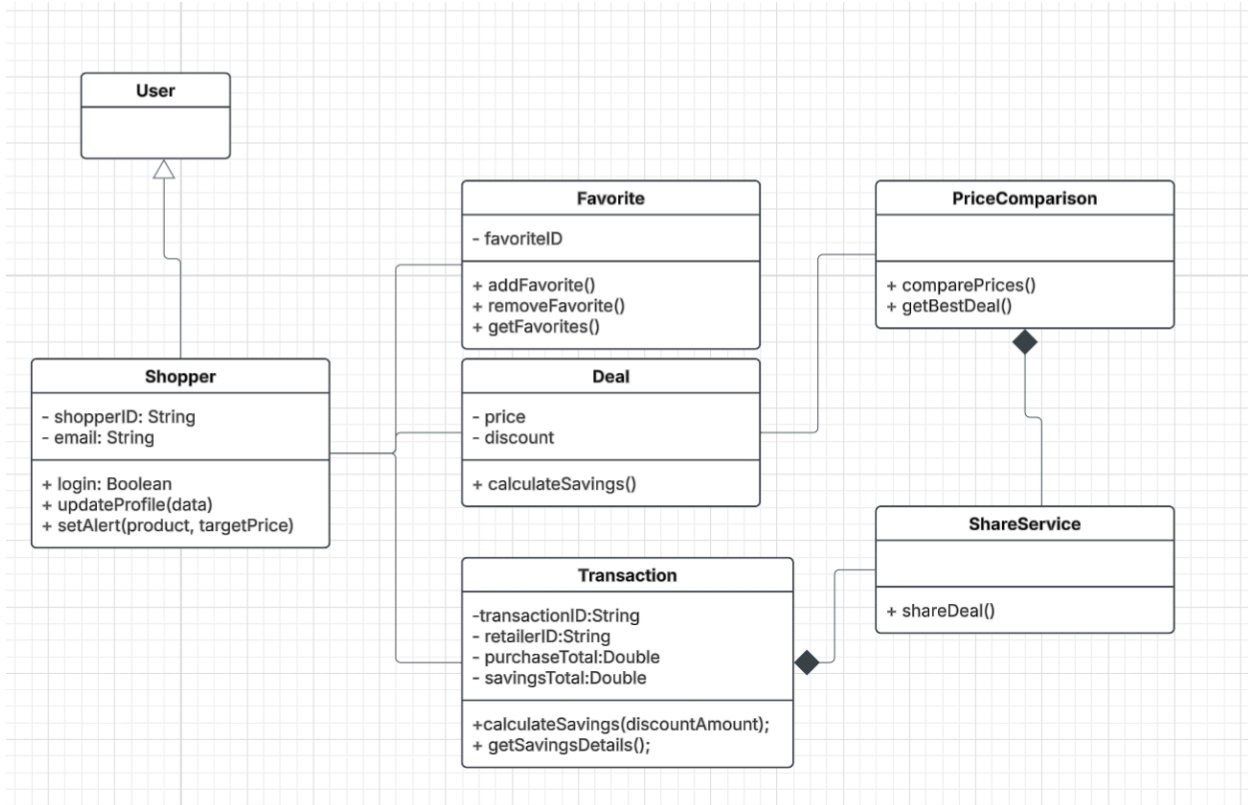
Use Case Descriptions:

1. Search for Product
2. Set Secure Price Alert
3. Filter/Sort Results

Elizabeth Philip:

- Diagrams



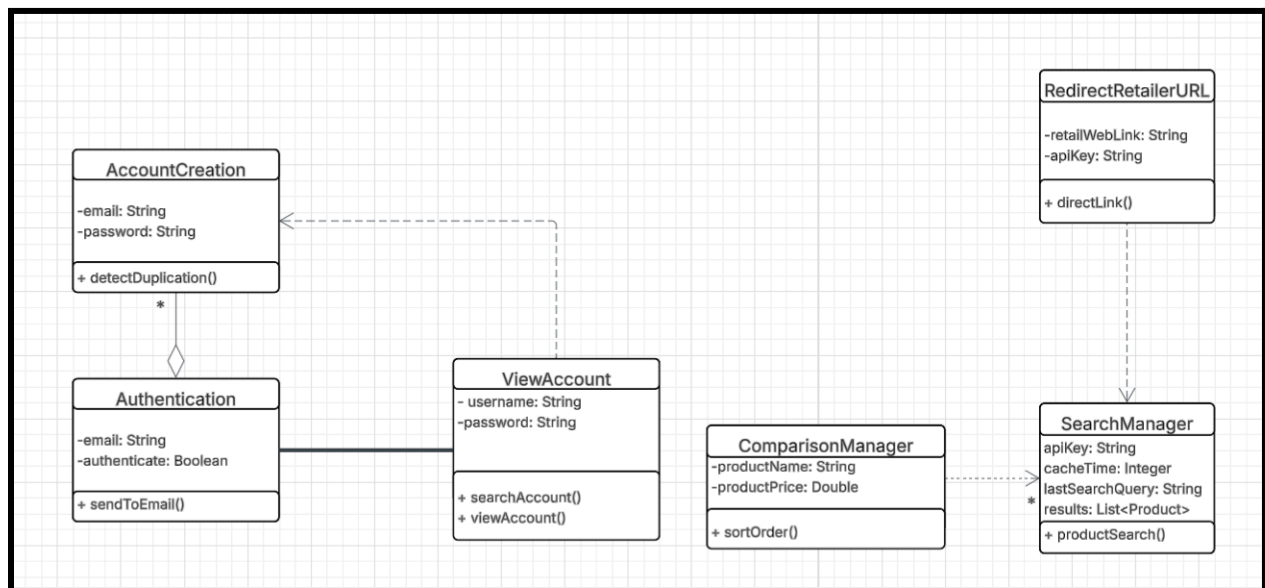
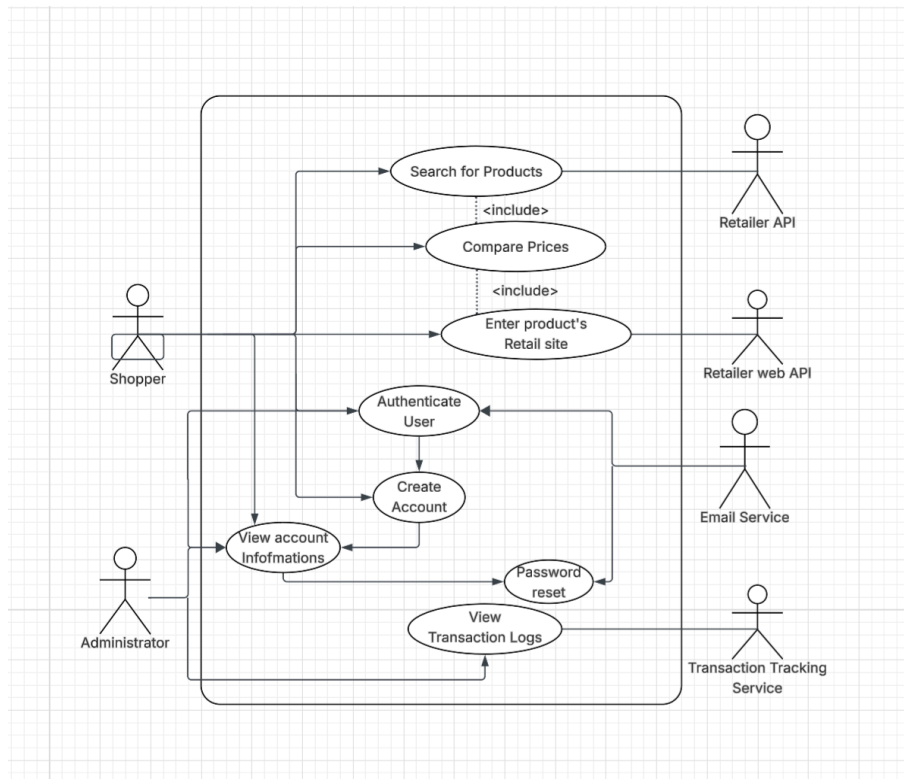


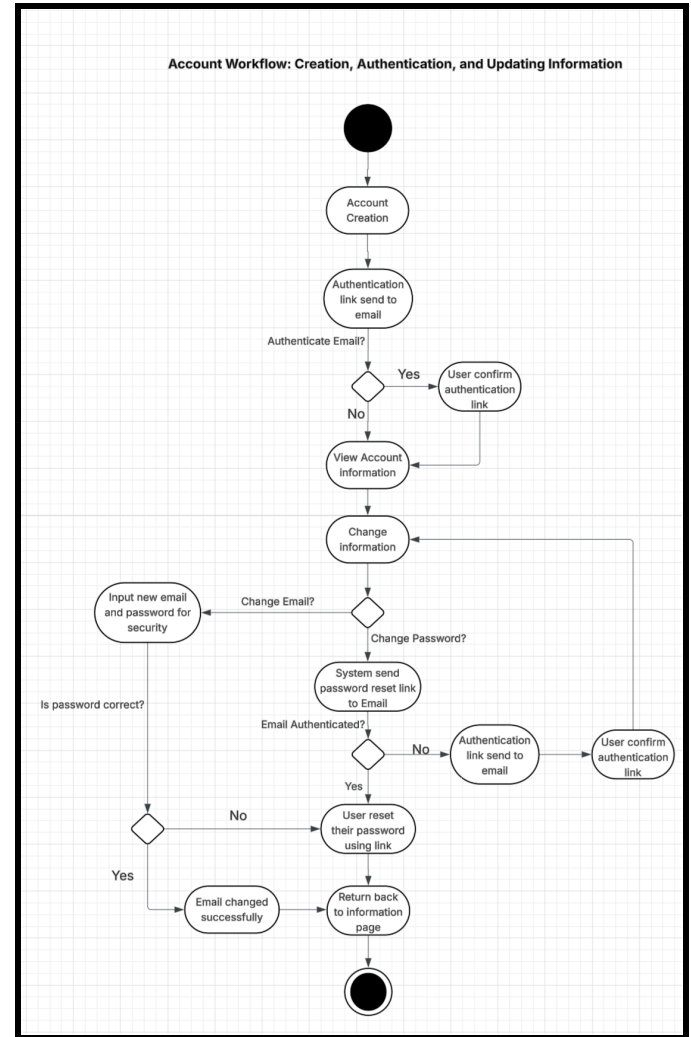
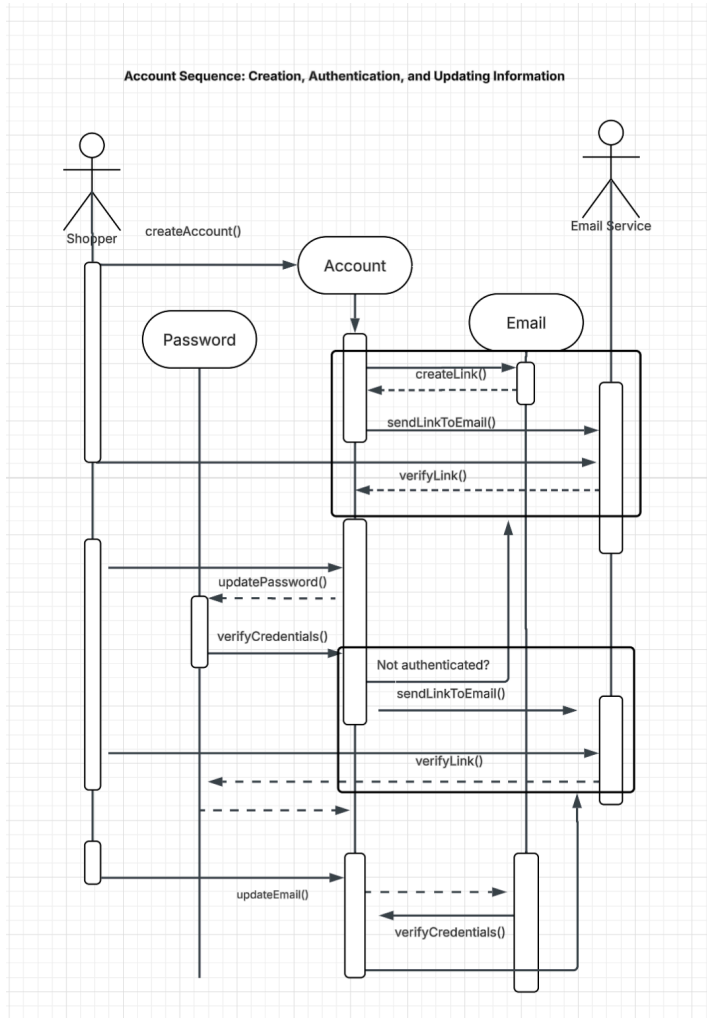
- **Use Case Descriptions:**

- View Savings History
- View Transaction Logs
- Share Deals

Xavier Le:

- Diagrams:





- **Use Case Descriptions:**
 - View User Account
 - Authenticate User
 - Redirect to Source URL